

Class 5: Classification

BUS 659

Prof. Jonathan Hersh

Exam, Oct 1-3

- Exam will be sent at 5pm on Oct 1, and due at 11:59pm on Oct 3rd.
- Exam will be take home, but I will be available to answer questions virtually
- Exam is open note and open book but do not discuss exam with other students, or use their answers or code
- Exam is structured like a long problem set with a mix of coding and conceptual questions.
- Exam is based on [take home challenges](#) common in data science interviews

Class 5: Outline

1. Why not regression for classification?
2. Logistic Function
3. Log Odds Ratio
4. Estimating Logistic Regressions in Python
5. Classification Lab 1
6. False/True Positives False/True Negatives
7. Confusion Matrices
8. ROC Curves and AUC
9. Classification Lab 2

Classification examples



How Do Businesses Use Classification Models?

- **Fraud Detection:** businesses classify transactions as fraudulent or legitimate to prevent financial losses.
- **Customer Churn Prediction:** identify customers at risk of leaving and take proactive retention steps.
- **Credit Scoring & Loan Approvals:** Classify applicants into low-risk vs. high-risk borrowers.
- **Healthcare Diagnosis:** Classify diseases, symptoms, or medical images for early detection.
- **Product Recommendations** – Classify user preferences to suggest the right products.
- **HR & Resume Screening** – Identify strong job candidates by analyzing past hiring trends.

Credit Card Default Dataset

Default rate: 0.0265

Student share: 0.3005

Rows: 4

	default	student	balance	income
0	No	No	1905.04	76094.34
1	No	No	1677.96	102957.10
2	No	No	1192.65	137960.58
3	No	No	1768.44	10000.00
4	No	Yes	1243.93	30979.30

Why not regression?

- Let's estimate a model predicting default based on credit card balance
- R² looks low but otherwise this model looks perfectly fine

```
df["default_binary"] = (df["default"].str.lower() == "yes").astype(int)

# OLS: default_binary ~ balance
ols_mod = smf.ols("default_binary ~ balance", data=df).fit()
print(ols_mod.summary())
```

OLS Regression Results						
=====						
Dep. Variable:	default_binary		R-squared:	0.037		
Model:	OLS		Adj. R-squared:	0.037		
Method:	Least Squares		F-statistic:	381.1		
Date:	Wed, 24 Sep 2025		Prob (F-statistic):	2.54e-83		
Time:	15:10:26		Log-Likelihood:	4285.0		
No. Observations:	10000		AIC:	-8566.		
Df Residuals:	9998		BIC:	-8552.		
Df Model:	1					
Covariance Type:	nonrobust					
=====						
	coef	std err	t	P> t	[0.025	0.975]

Intercept	-0.0414	0.004	-10.844	0.000	-0.049	-0.034
balance	5.61e-05	2.87e-06	19.521	0.000	5.05e-05	6.17e-05

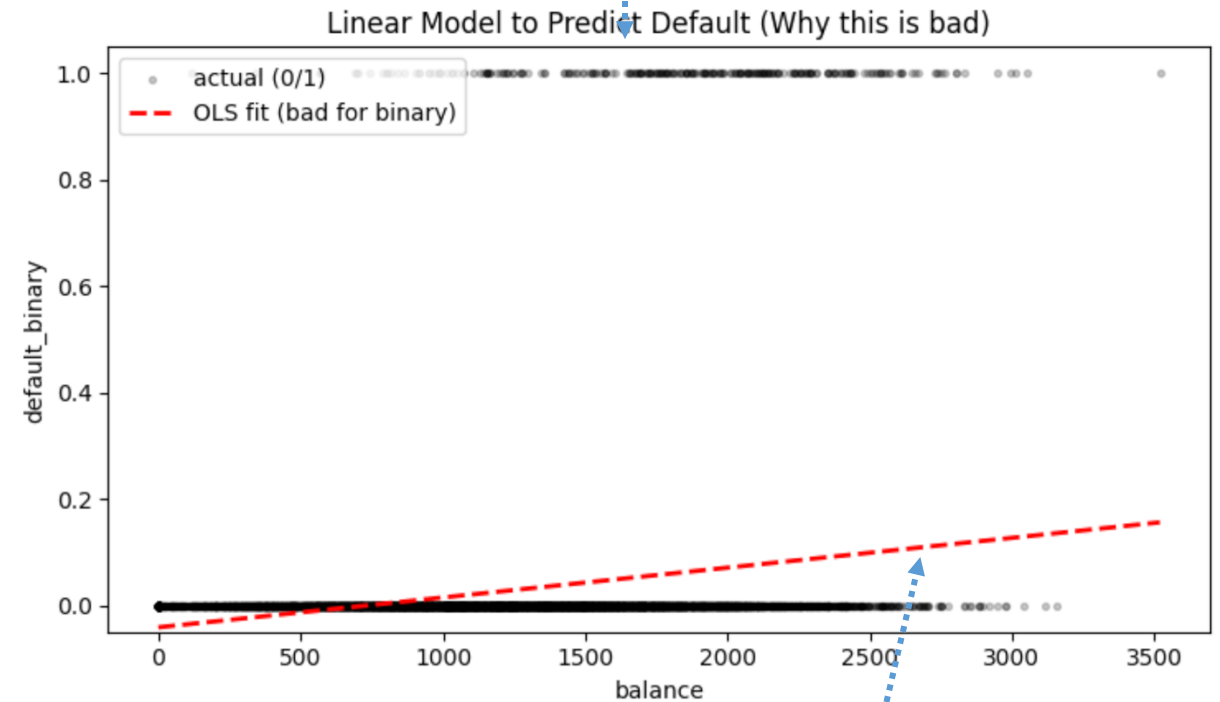
Linear Model to Predict Default

```
# Predictions dataframe
preds_DF = pd.DataFrame({
    "preds": ols_mod.predict(df),
    "balance": df["balance"].values,
    "default_binary": df["default_binary"].values
})
print(preds_DF.head())

# Build regression line from coefficients
intercept = ols_mod.params["Intercept"]
slope = ols_mod.params["balance"]
```

- Let's use the model to generate predictions of default (which is binary)

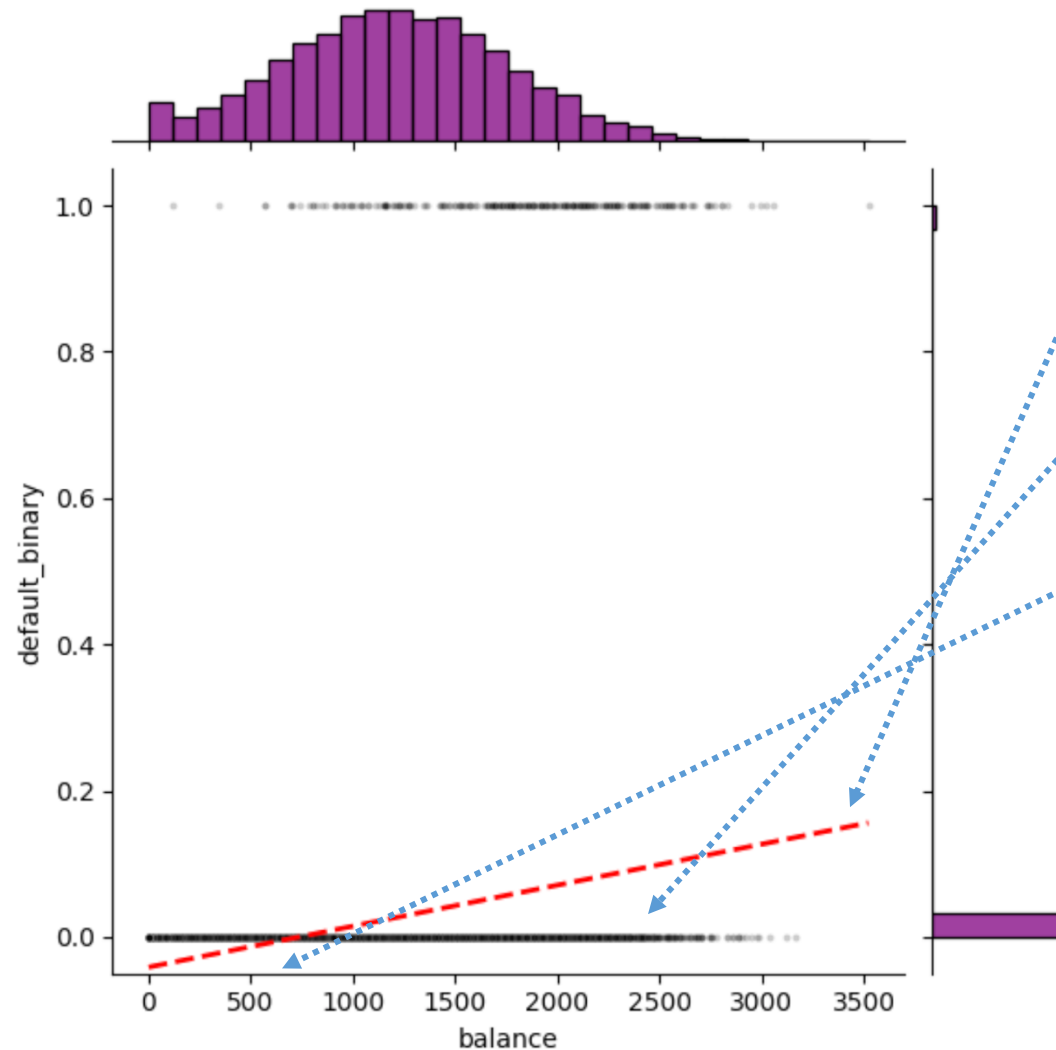
- Black dots show actual default behavior



- Red line shows the predictions from the model

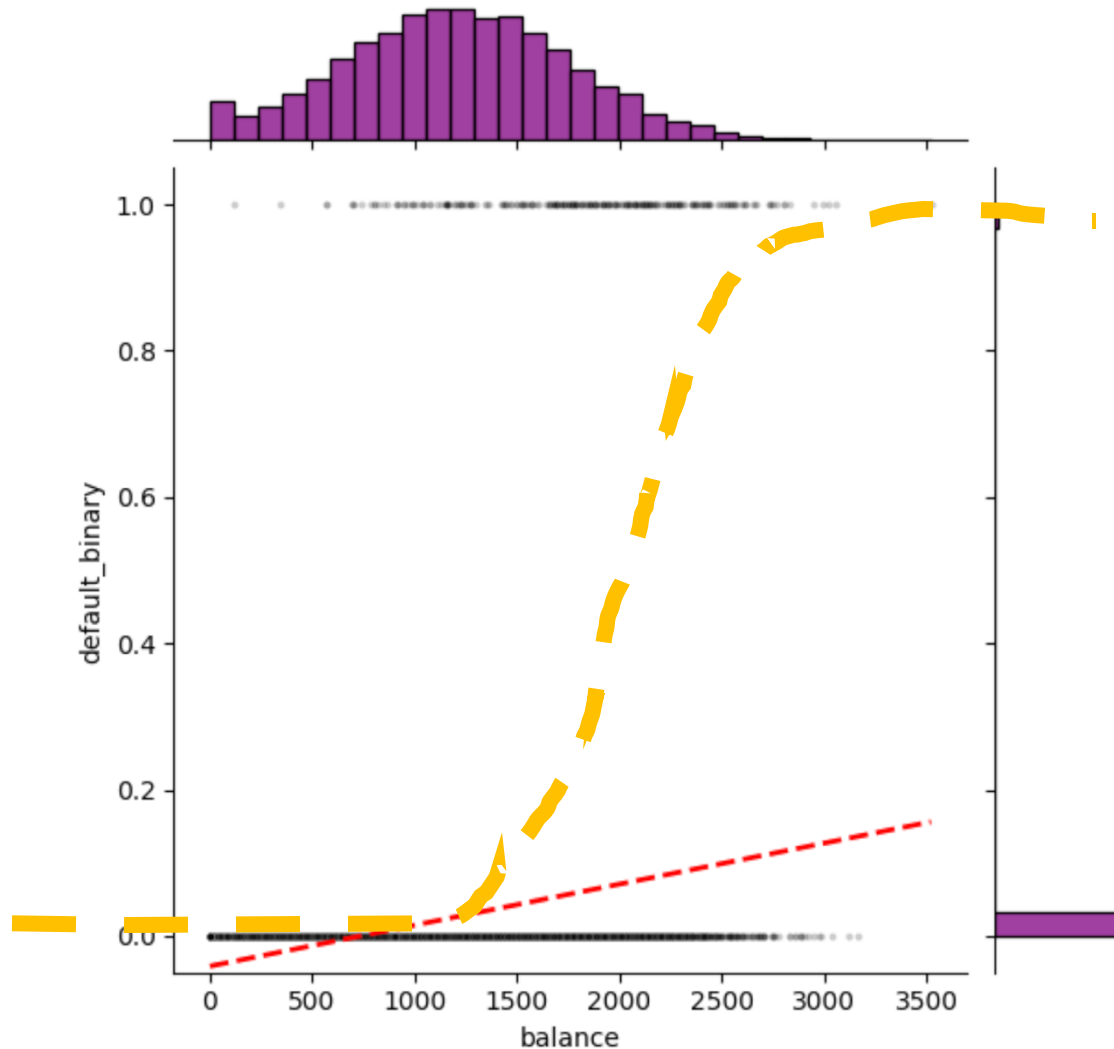
Why is the red line a bad prediction model for default?

Why Is This a Bad Prediction Model?



1. We never predict more than 30% chance of default!
2. Most observations do not default!
3. We predict negative probability of default!

One Way to Improve the Earlier Model: Compress Predictions to Lie Between 0 and 1



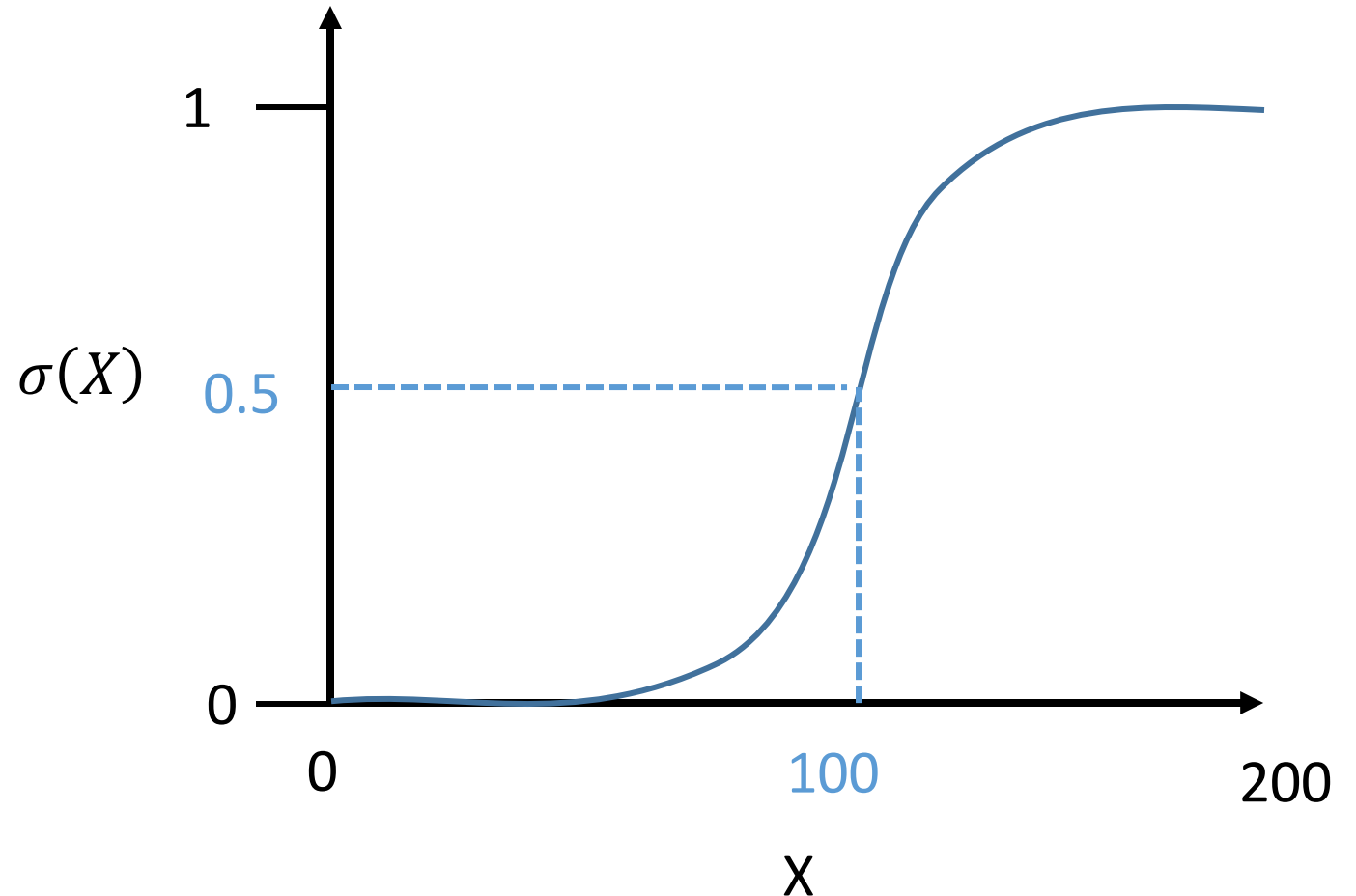
- Because probabilities are between 0 and 1 we want to compress red line to lie within 0 and 1 on the y axis
- i.e let $p(X) = Pr(Y = 1|X)$ be the probability the event occurs
- We want our model to output:

$$Pr(Y = 1|X) \in [0,1]$$

What is The Logistic/Sigmoid Function

- Logistic is a function that naturally takes inputs X and transforms between 0 and 1
- The logistic is defined as

$$\sigma(X) = \frac{1}{1 + e^{-X}} = \frac{e^X}{e^X + 1}$$



Using the Logistic/Sigmoid Function to Generate Probabilities

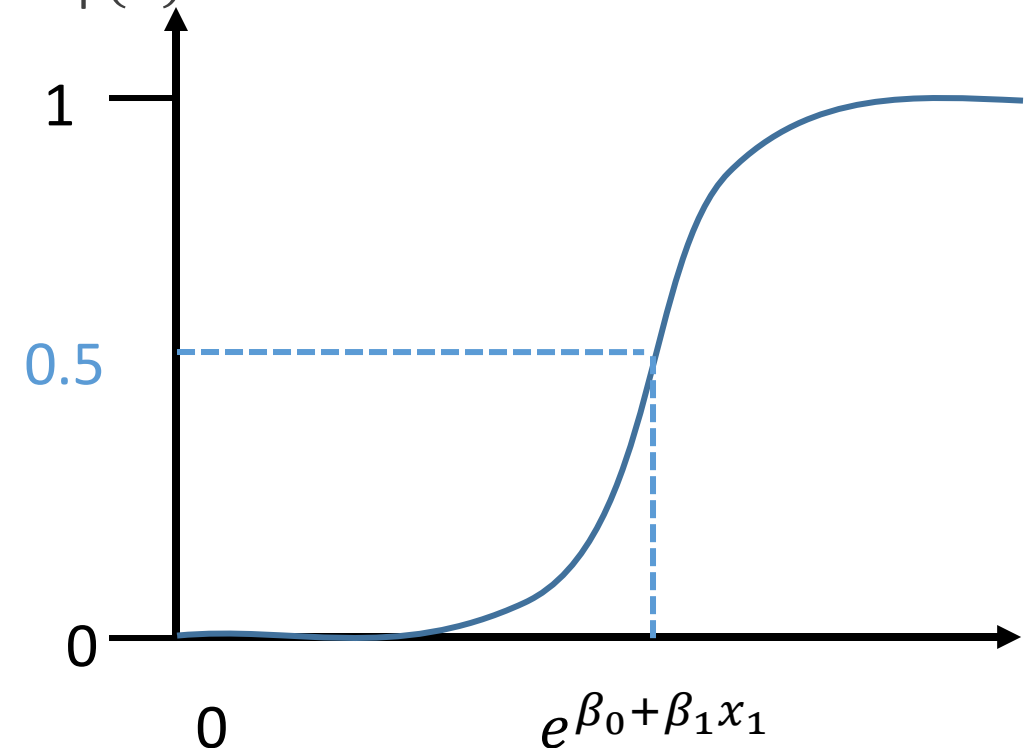
- How do we generate probabilities from this function?
- We let $X = \beta_0 + \beta_1 \cdot x_1 + \dots + \beta_k \cdot x_k$ and plug this into the logistic function

$$\sigma(X) = \frac{1}{1 + e^{-X}} = \frac{e^X}{e^X + 1}$$

$$Pr(Y = 1|X) = \frac{e^{\beta_0 + \beta_1 \cdot x_1}}{e^{\beta_0 + \beta_1 \cdot X} + 1}$$

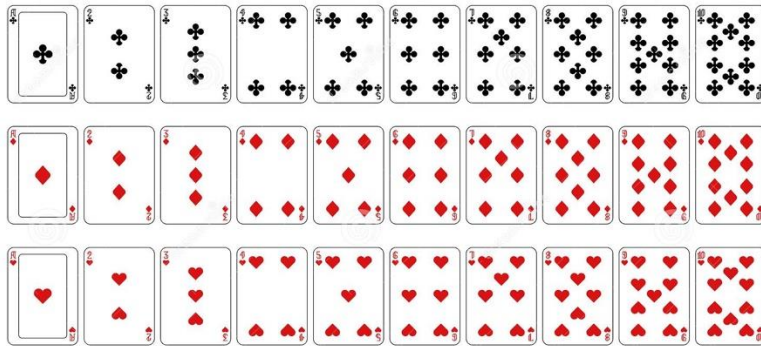
$$Pr(Y = 1|X) = \frac{1}{1 + e^{-(\beta_0 + \beta_1 x_1)}}$$

Probability of event
happening i.e $p(X)$



This is equivalent
mathematically! I promise.
Work it out on pen and paper
if you don't believe me

What is the “Odds Ratio”



- The outcome variable in a logit regression is the “odds ratio” (OR)
- In a deck of 52 cards there are 13 spades
- The probability of randomly drawing a spade is $13/52 = 25\%$
- The probability of not drawing a spade is $39/52 = 75\%$
- Therefore the ratio of odds of drawing a spade vs not drawing a spade is

$$\frac{\text{ratio of drawing a spade}}{\text{ratio of not drawing a spade}} = \frac{13/52}{39/52} = \frac{13}{39} = 1:3 = 0.333$$

- Log odds ratio is just $\log(0.333) = -0.4771\dots$

Logit Models Model the Outcome As a Log Odds Ratio

$$\frac{p(Y=1|X)}{p(Y=0|X)} = e^{\beta_0 + \beta_1 X}$$

$$\log \left(\frac{p(Y=1|X)}{p(Y=0|X)} \right) = \log(e^{\beta_0 + \beta_1 X})$$

$$\log \left(\frac{p(Y = 1|X)}{p(Y = 0|X)} \right) = \beta_0 + \beta_1 X$$

The outcome variable (Y) for a logistic regression is the log odds ratio

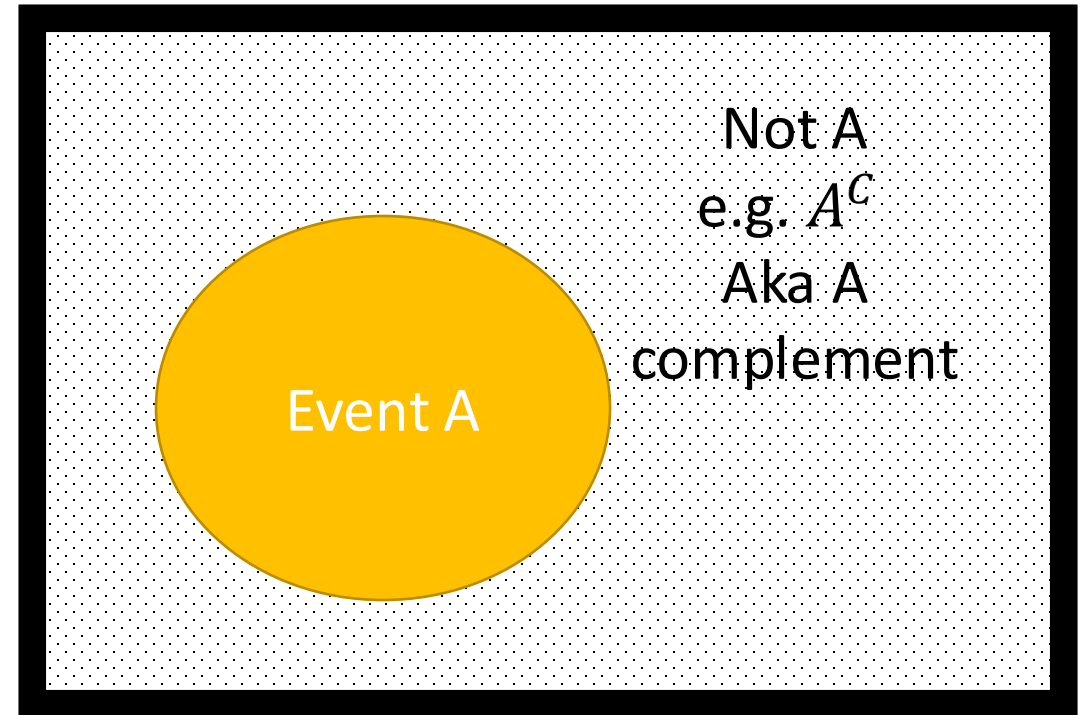
Log odds ratio is a linear expression of the X-variable's impact in a nonlinear manner

All logistic coefficients can be interpreted as that variable's impact on log odds ratio

Probability Note on The Complement

- Q : if $\Pr(A) = 30\%$
- What is the probability of A not happening (the complement) or $\Pr(A^C)$?
- Because events A and not A fully partition the sample space
$$\Pr(A^C) = 1 - P(A)$$
- Fully partition the sample space (i.e. two events are all that can happen):
$$A \cup A^C = \Omega = 1$$

Sample Space (All possible outcomes)



How Do We Find $P(Y=0)$? Recall Complement Trick

Probability the
Event Happens

$$\Pr(Y = 1|X) = \frac{e^{\beta_0 + \beta_1 X}}{1 + e^{\beta_0 + \beta_1 X}}$$

Probability the
Event Does not
Happen!
Complement
Trick

$$\Pr(Y = 0|X) = 1 - \Pr(Y = 1|X)$$

$$\Pr(Y = 0|X) = 1 - \frac{e^{\beta_0 + \beta_1 X}}{1 + e^{\beta_0 + \beta_1 X}}$$

$$\Pr(Y = 0|X) = \frac{1 + e^{\beta_0 + \beta_1 X}}{1 + e^{\beta_0 + \beta_1 X}} - \frac{e^{\beta_0 + \beta_1 X}}{1 + e^{\beta_0 + \beta_1 X}}$$

$$\Pr(Y = 0|X) = \frac{1 + e^{\beta_0 + \beta_1 X} - e^{\beta_0 + \beta_1 X}}{1 + e^{\beta_0 + \beta_1 X}} = \frac{1}{1 + e^{\beta_0 + \beta_1 X}}$$



Expressing Ratio of Probabilities: Odds Ratio

- We now have expressions for $P(Y=1)$ and $P(Y=0)$ events
- Some algebra (division is equal to multiplication of the reciprocal)
- Next we can cross out $1 + e^{\beta_0 + \beta_1 X}$

• What this equation shows is that if we exponentiate the coefficients we can interpret the coefficient as the impact of the x variable on the odds ratio directly!

$$\begin{aligned} \frac{p(Y = 1|X)}{p(Y = 0|X)} &= \frac{e^{\beta_0 + \beta_1 X}}{1 + e^{\beta_0 + \beta_1 X}} / \frac{1}{1 + e^{\beta_0 + \beta_1 X}} \\ &= \frac{e^{\beta_0 + \beta_1 X}}{1 + e^{\beta_0 + \beta_1 X}} \cdot \frac{1 + e^{\beta_0 + \beta_1 X}}{1} \\ &= \frac{e^{\beta_0 + \beta_1 X}}{\cancel{1 + e^{\beta_0 + \beta_1 X}}} \cdot \frac{\cancel{1 + e^{\beta_0 + \beta_1 X}}}{1} \end{aligned}$$

$$\underbrace{\frac{p(Y = 1|X)}{p(Y = 0|X)}}_{\text{Odds ratio}} = \underbrace{e^{\beta_0 + \beta_1 X}}_{\text{Exponential of our coefficients}}$$

Estimating Logit Models Using `smf.logit()`

```
# Logistic regression: default ~ student
logit_fit1 = smf.logit("default_binary ~ student_bin",
| | | | | | data=df).fit()
print(logit_fit1.summary())
```

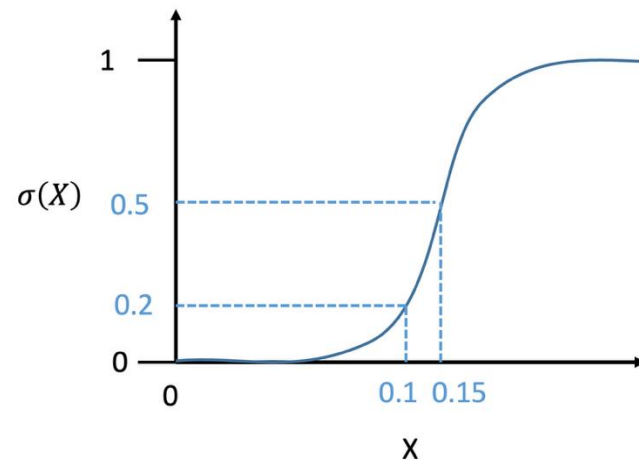
Logit Regression Results						
=====						
Dep. Variable:	default_binary		No. Observations:		10000	
Model:	Logit		Df Residuals:		9998	
Method:	MLE		Df Model:		1	
Date:	Wed, 24 Sep 2025		Pseudo R-squ.:		0.01496	
Time:	15:29:33		Log-Likelihood:		-1205.3	
converged:	True		LL-Null:		-1223.6	
Covariance Type:	nonrobust		LLR p-value:		1.449e-09	
=====						
	coef	std err	z	P> z	[0.025	0.975]

Intercept	-3.3950	0.067	-50.314	0.000	-3.527	-3.263
student_bin	-0.9627	0.177	-5.450	0.000	-1.309	-0.616
=====						

- Estimate logistic regression using the function `smf.logit()`
- We still specify a formula in the usual manner $y \sim x$
- Coefficients are estimated by Maximum Likelihood, not least squares.
- The fitted model returns predicted probabilities via `.predict()`.

Null hypothesis for logistic model is $\beta = 0$!

	coef	std err
Intercept	-3.3950	0.067
student_bin	-0.9627	0.177



- What do the coefficients tell us?
- When $\beta > 0$ **increasing** this variable makes it **more likely** that the event we are trying to predict occurs
- When $\beta < 0$ **increasing** this variable makes it **less likely** that the event we are trying to predict occurs
- Note: the **impact of β on y is non-linear**. Meaning small changes in x can result in big changes in $P(y=1)$, or small changes in x can have big impacts on $P(y=1)$

Estimating Impact of Being a Student on Default Probability

$$\log\left(\frac{p(Y = \text{default}|X)}{p(Y = \text{not default}|X)}\right) = \beta_0 + \beta_1 \cdot \text{student}_i + \epsilon_i$$

$$\exp\left(\log\left(\frac{p(Y = \text{default}|X)}{p(Y = \text{not default}|X)}\right)\right) = \exp(\beta_0 + \beta_1 \cdot \text{student}_i + \epsilon_i)$$

$$\frac{p(Y = \text{default}|X)}{p(Y = \text{not default}|X)} = \exp(\beta_0 + \beta_1 \cdot \text{student}_i + \epsilon_i)$$

	coef	std err
Intercept	-3.3950	0.067
student_bin	-0.9627	0.177

```
np.exp(logit_fit1.params['student_bin'])
```

✓ 0.0s

0.38185681260384247

- Remember the outcome variable in a logistic regression is the **log odds ratio**
- If we exponentiate the coefficients this tells us the impact of the variable on the unlogged odds ratio
- If we take our estimated logistic model we see $\beta_1 = -0.9627$, $\exp(\text{beta1}) = 0.38$
- Odds ratio of **0.38** means: the odds of default for students are **only 38% as large** as the odds for non-students.
- Equivalently, students have a **62% reduction in odds of default:**

Lab Time!

Lab 1 — Logistic Regression in Python

Dataset has columns:

- `default` (Yes/No)
- `student` (Yes/No)
- `balance` (numeric)
- `income` (numeric)

We'll estimate a logit model and interpret coefficients, p-values, and odds ratios.

1) Estimate the model

Estimate:

`default_binary ~ balance + income + student_bin`

Store the fitted model as `logit_mod2`.

✓

]

Generating Predicted Probabilities from a Logit Model

- To generate predictions, we use the estimated coefficients in the logit equation

$$\hat{p}(X = 1000) = \frac{e^{\hat{\beta}_0 + \hat{\beta}_1 x_1}}{1 + e^{\hat{\beta}_0 + \hat{\beta}_1 x_1}} =$$

- The estimated probability of default with a balance of \$1,000 is given by
- The estimated probability of default with a balance of \$2,000 is given by

- To generate predicted probability for all observations in a dataset we use the predict method.

- This is also called “scoring” a dataset

Model:	Logit	Method:	MLE
Dependent Variable:	default_binary	Pseudo R-squared:	0.151
Date:	2025-09-24 16:05	AIC:	2080.7875
No. Observations:	10000	BIC:	2095.2082
Df Model:	1	Log-Likelihood:	-1038.4
Df Residuals:	9998	LL-Null:	-1223.6
Converged:	1.0000	LLR p-value:	1.5698e-82
No. Iterations:	9.0000	Scale:	1.0000

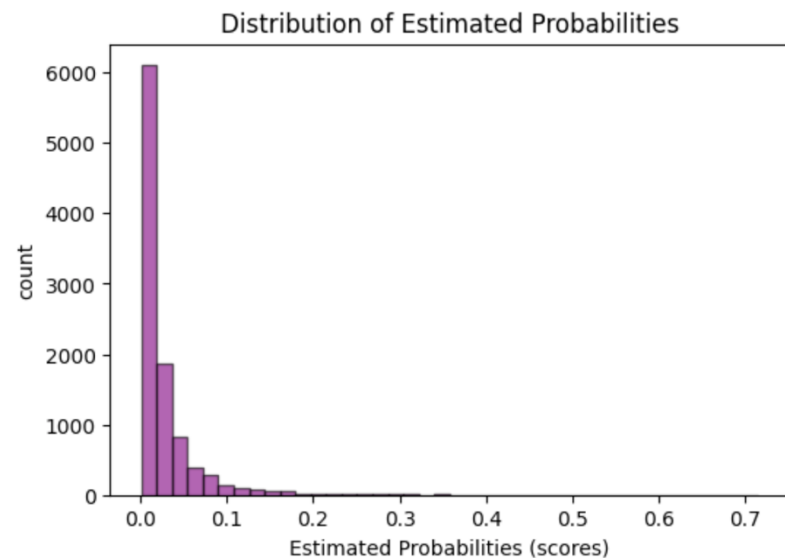
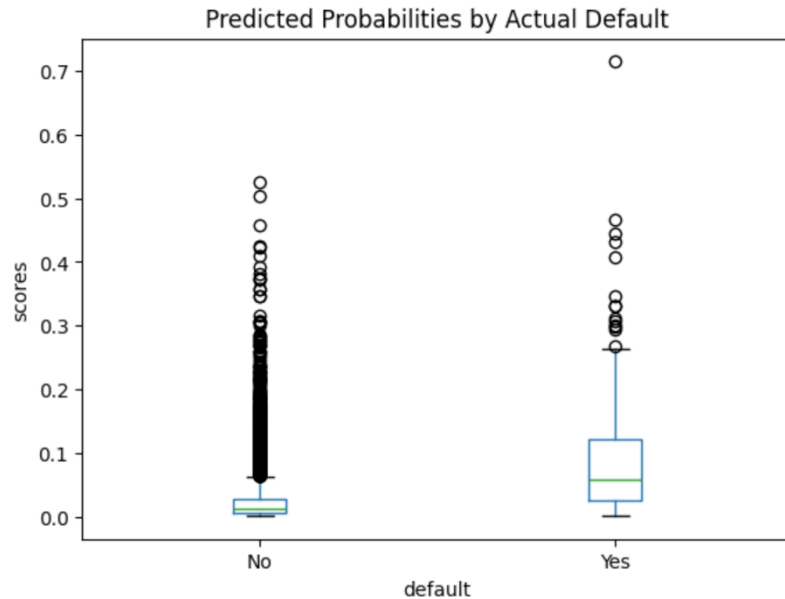
	Coef.	Std.Err.	z	P> z	[0.025	0.975]
Intercept	-7.0538	0.2358	-29.9109	0.0000	-7.5160	-6.5916
balance	0.0023	0.0001	17.9727	0.0000	0.0020	0.0025

$$\hat{p}(X = 1000) = \frac{e^{-7.0538 + 0.0023 \cdot 1000}}{1 + e^{-7.0538 + 0.0023 \cdot 1000}} = 0.0085$$

$$\hat{p}(X = 2000) = \frac{e^{-7.0538 + 0.0023 \cdot 2000}}{1 + e^{-7.0538 + 0.0023 \cdot 2000}} = 0.079$$

```
# Get predicted probabilities for the training data  
scores = logit_fit3.predict()
```

What Do We Do With Scores or Estimated Probabilities?

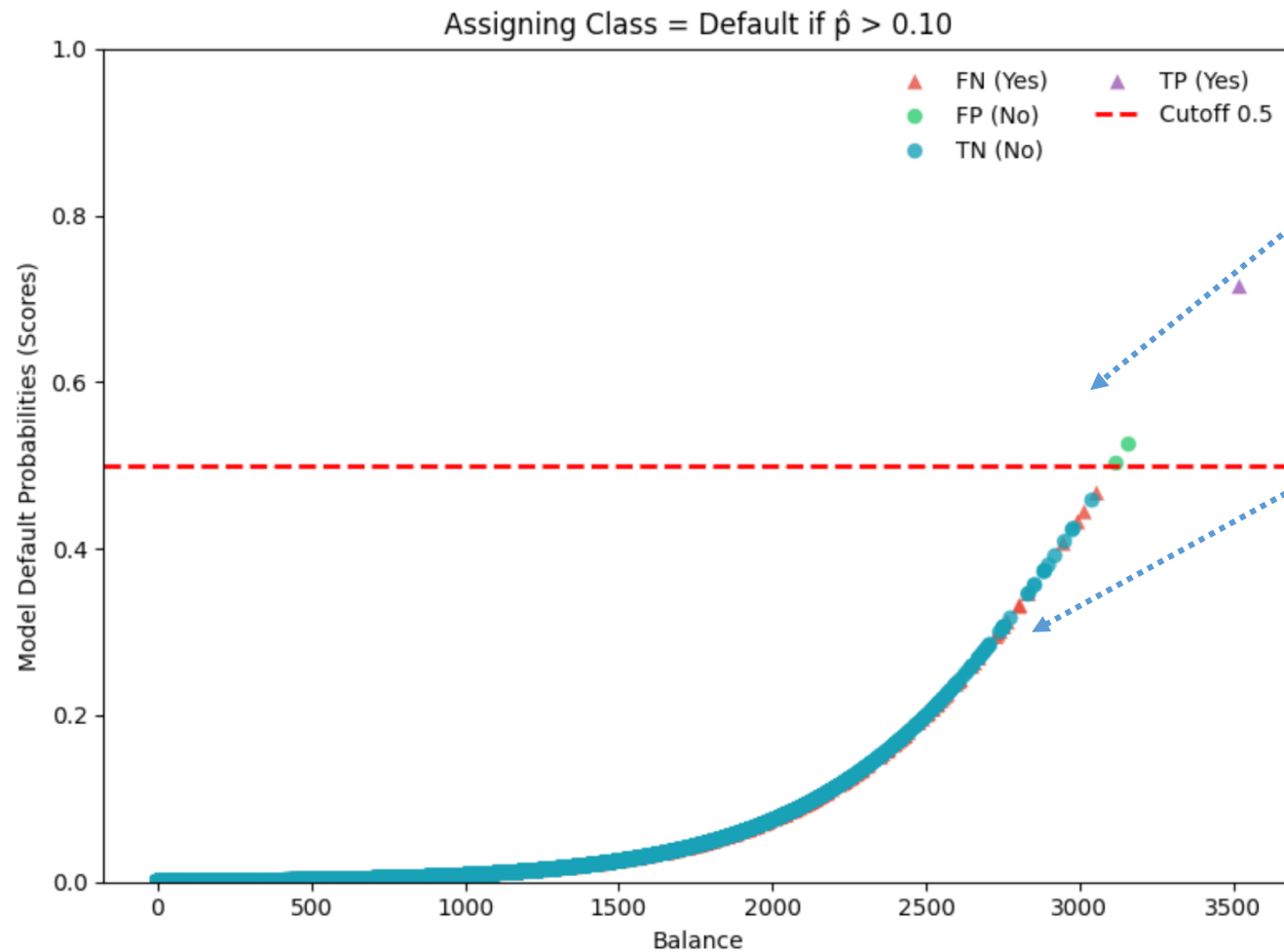


- Okay, so we have probabilities, what then?
- Note there is almost always some overlap between the probabilities of the classes
- We can't choose a probability such that above this all actual defaulters are correctly identified, and below this all non-defaulter are identified
- So we will always have some **false positives** and **false negatives**

Confusion Matrix: Table of False/True Positives and False/True Negatives

		True default status	
		No	Yes
Predicted default status	No	True negative (TN)	False Negative (FN)
	Yes	False Positive (FP)	True Positive (TP)

Assigning Class=Default to $\hat{p} > 0.5$

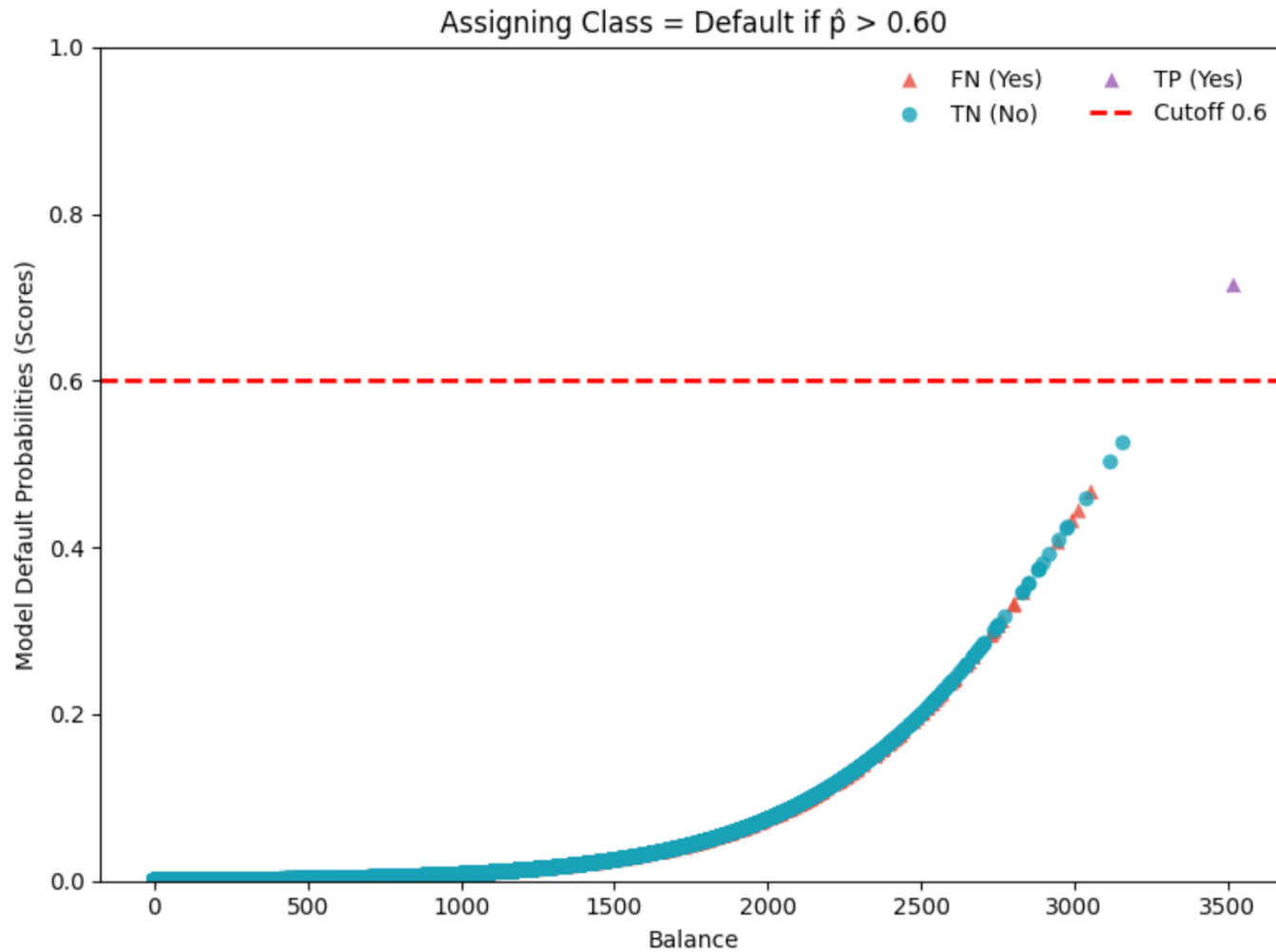


FN=264, FP=2, TN=9733, TP=1

- Above this line, observations are classified as defaulting
- Below this line, observations classified as **not** defaulting
- Actual default = teal triangles
- If we choose a probability cutoff of 0.5, then we see we have no one classified as default in our model
- This is often the case with low probability events

Note I'm working with a 5% sample of the dataset to make the numbers easier

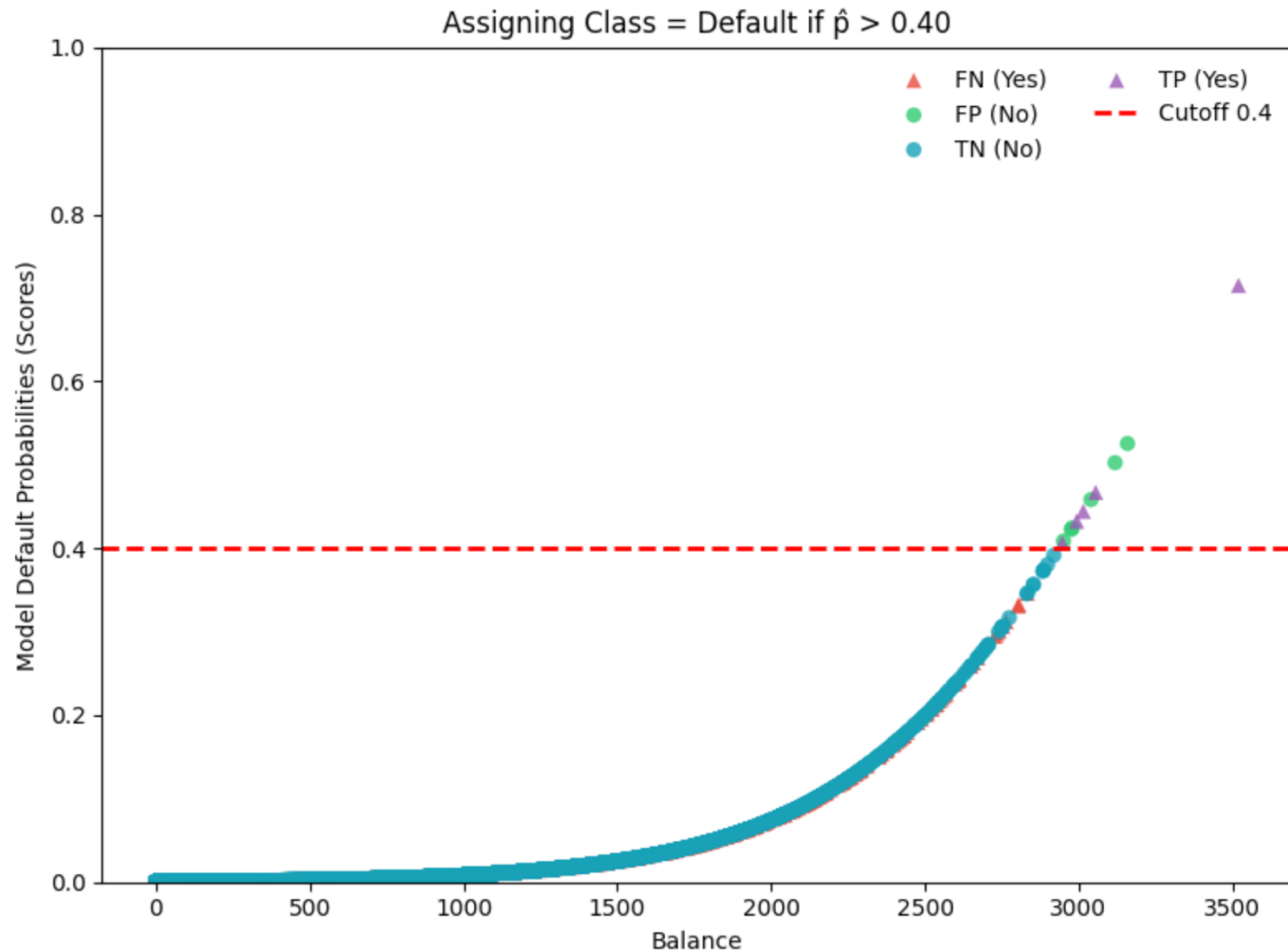
Assigning Class=Default to $\hat{p} > 0.6$



- Raising the cutoff to 0.6, then we see we have 0 false positives and 264 FN

FN=264, FP=0, TN=9735, TP=1

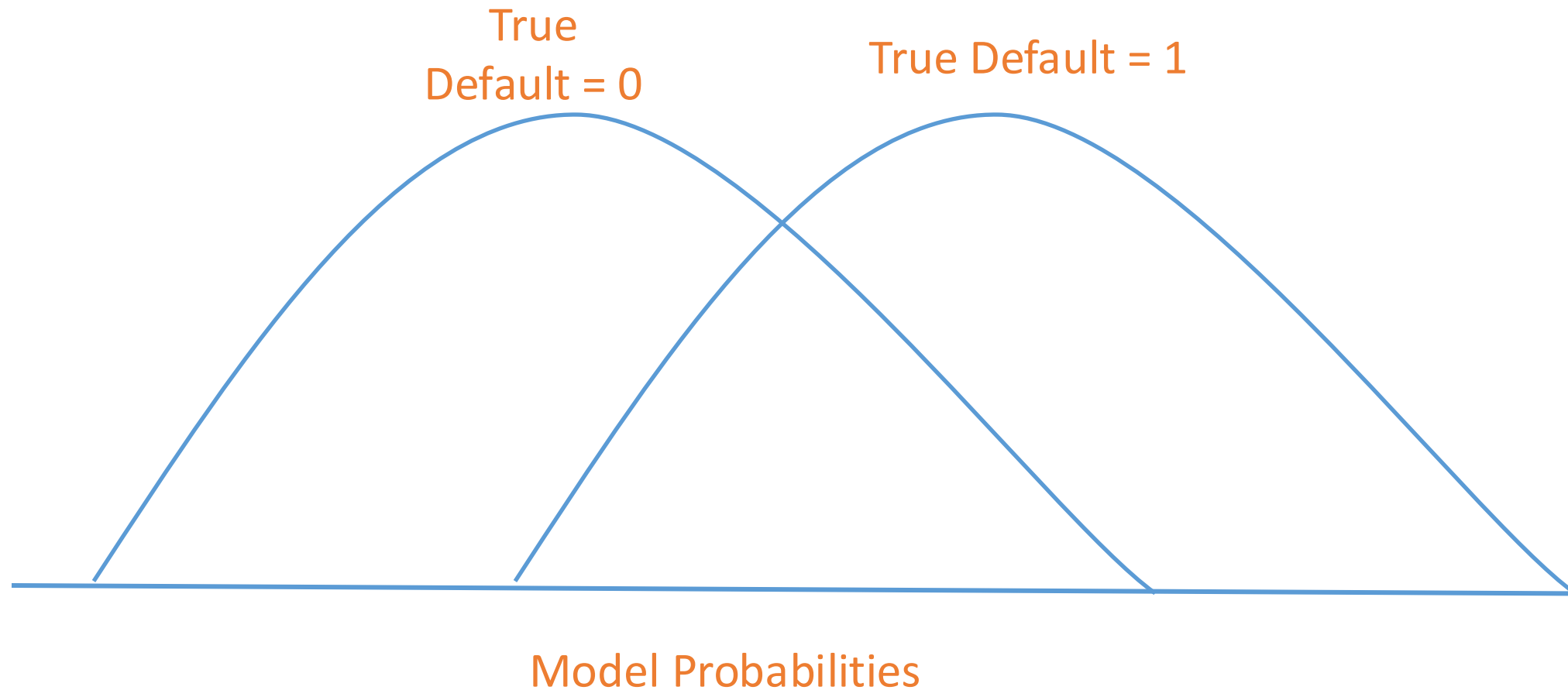
Assigning Class=Default to $\hat{p} > 0.4$



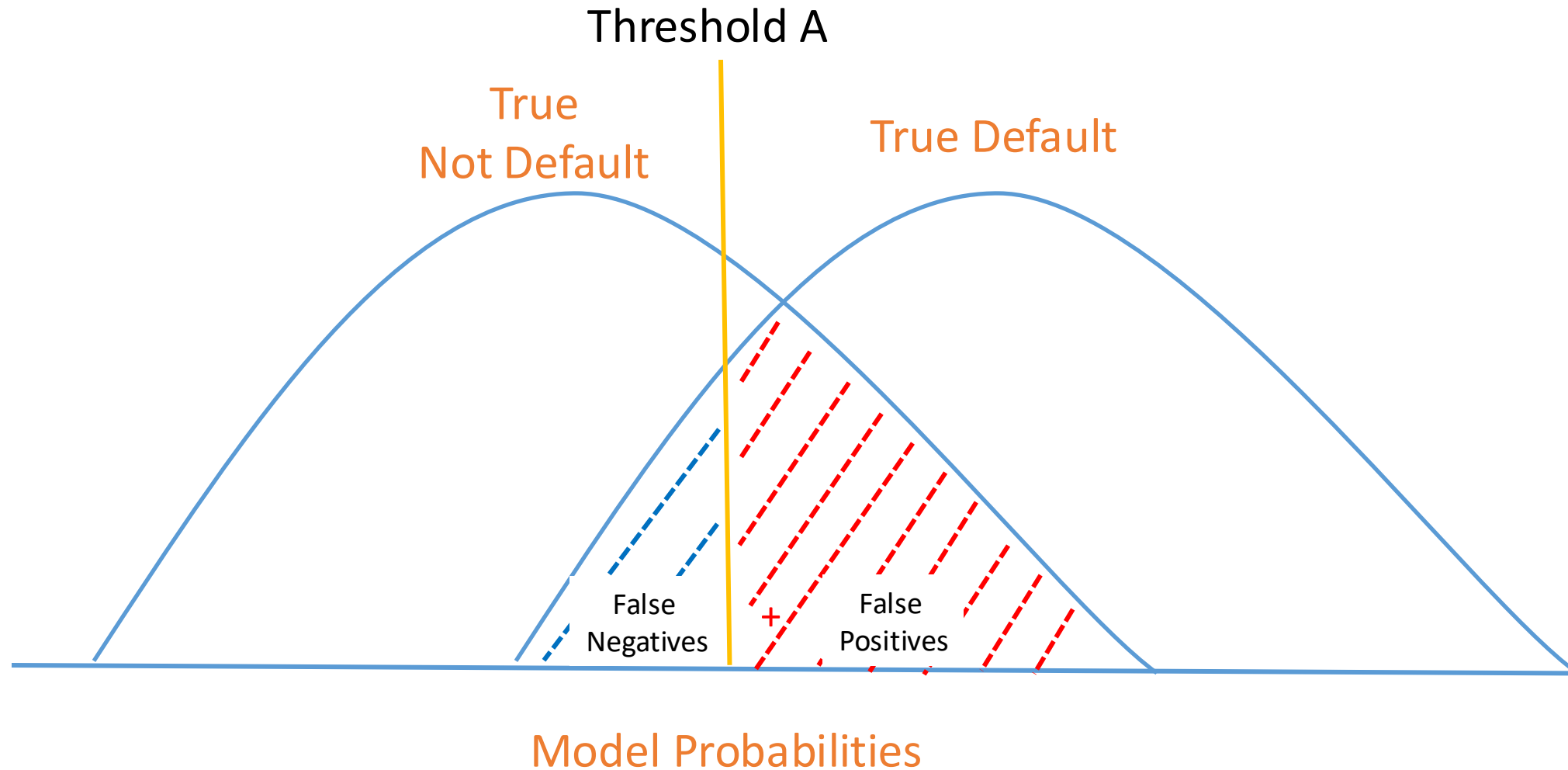
- Lowering the cutoff to 0.4 results in more FPs (6) but fewer FNs (260)

FN=260, FP=6, TN=9729, TP=5

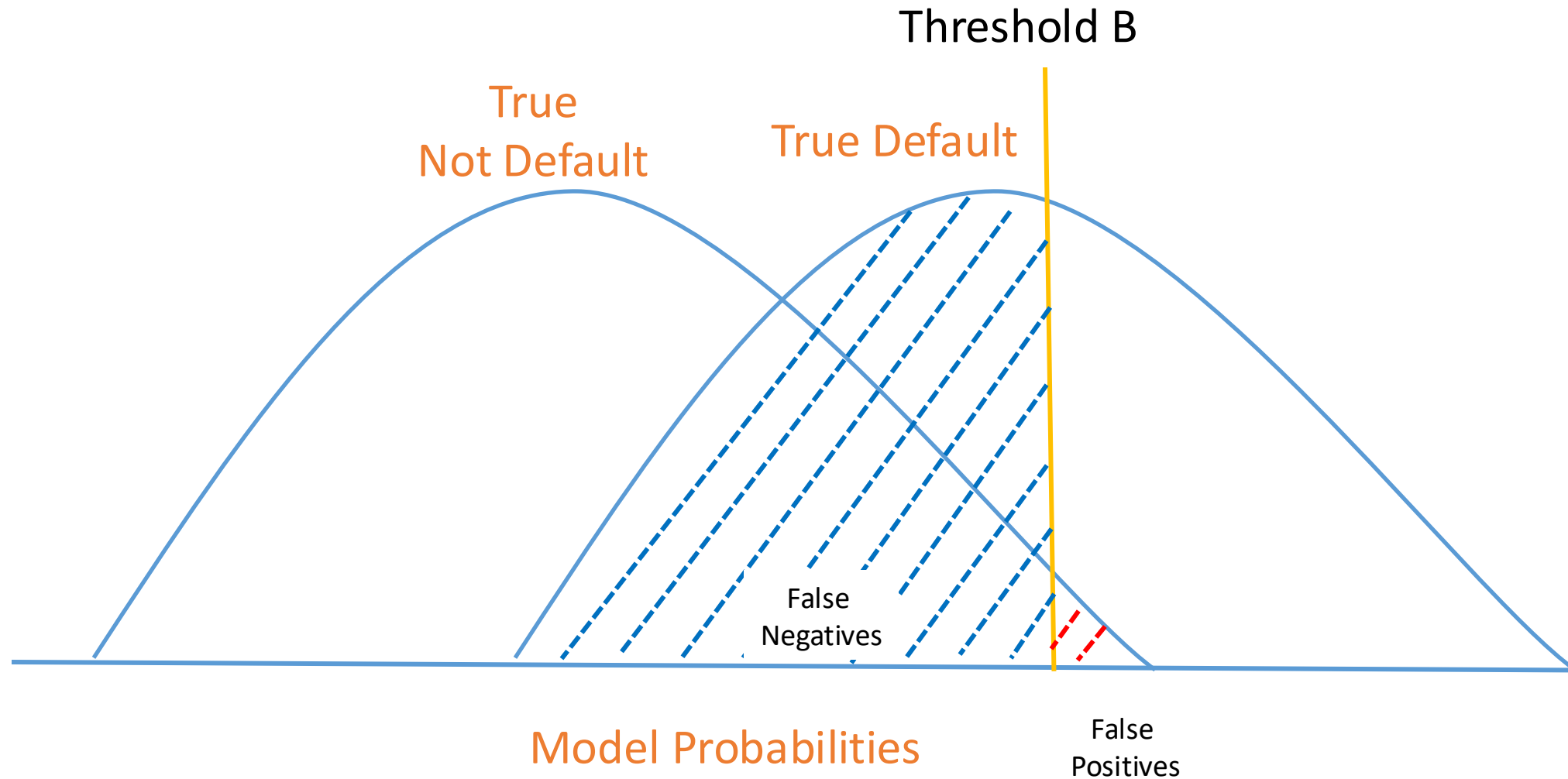
Choosing Probability Cutoff to Assign Class



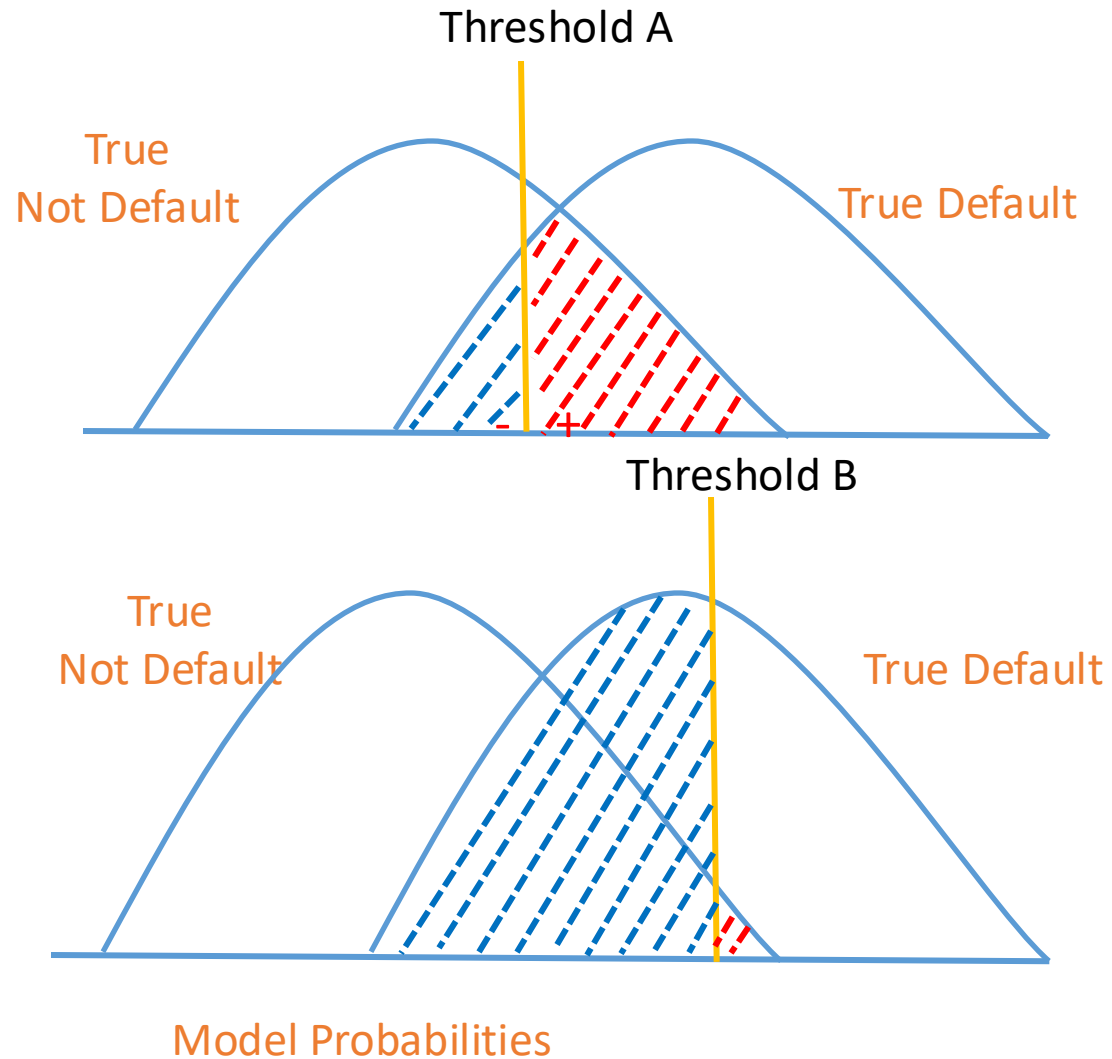
Threshold A: Moderate Threshold



Threshold B: Higher Threshold



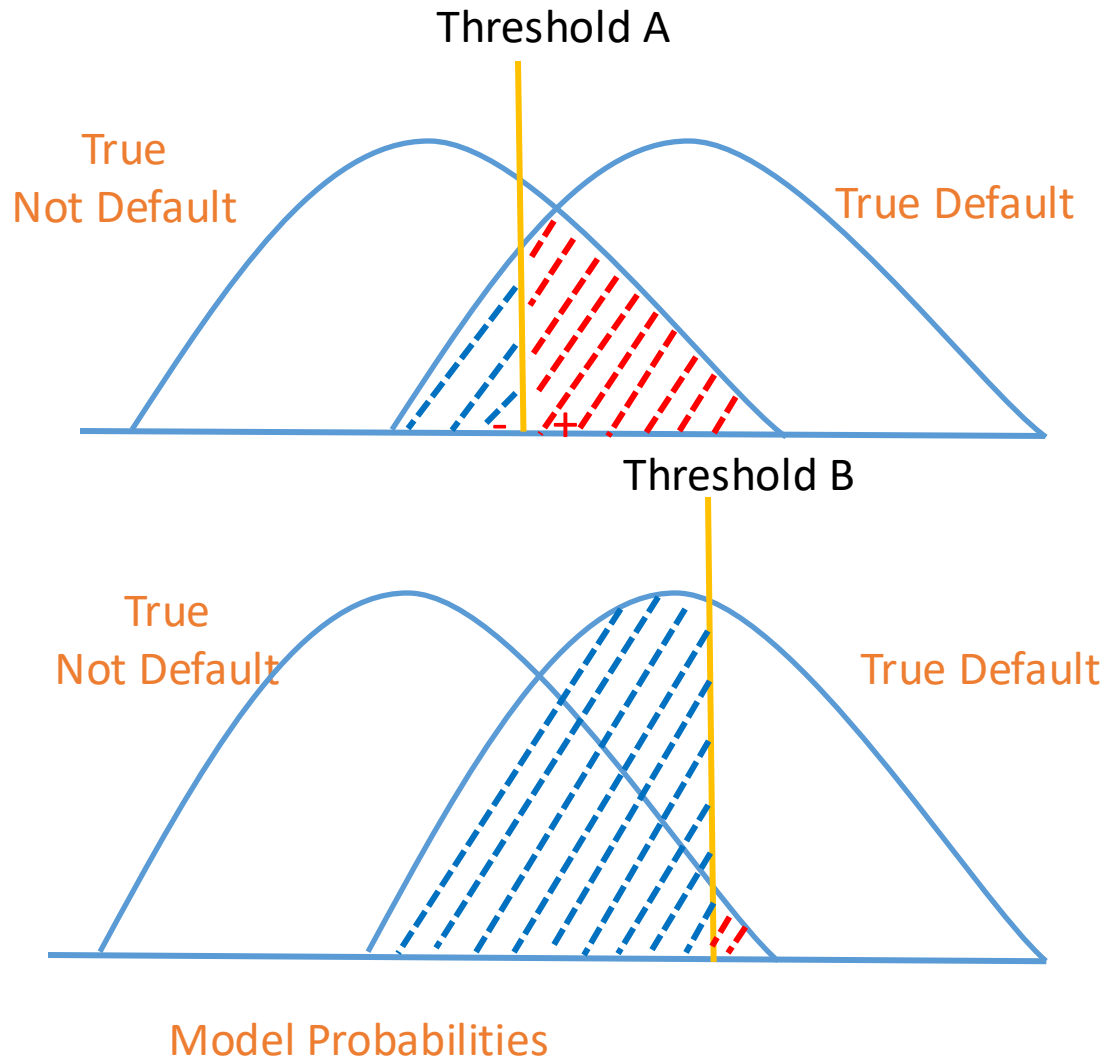
Comparing cutoffs:



Many false positives

Few false positives

Which Probability Cutoff To Use?



- Threshold you choose should depend on relative costs of FPs and FN
 - e.g. screening at airport (cost of false neg high)
 - e.g. direct mail advertisement (cost of false positive low)
- Some common choices
 - **Maximize Accuracy** (equal weighting of FPs and FNs)
 - **Threshold $\hat{p} > 0.5$**
 - **Minimize cost:** $TC = \text{costFP} * \text{FPs} + \text{costFN} * \text{FNs}$

From Models to Decisions: Using Classification in Business Strategy

- A well-designed classification model provides managers with actionable insights.
- **False Positive and False Negative tradeoffs:**
 - Managers must decide what's costlier (e.g., approving a risky loan vs. rejecting a good customer).
 - Threshold Adjustment → Should we be more strict or lenient in classification?
- **Marketing:** Who should get personalized promotions? (Loyal vs. At-risk customers)
- **Operations:** Which orders are likely to be delayed? (On-time vs. Late shipments)
- **Takeaway:** Classification, particularly logistic regression, is a decision-support tool that helps businesses predict and categorize outcomes based on data-driven probabilities.
- By modeling the likelihood of events such as customer churn, fraud detection, or loan default, **classification enables managers to make proactive, risk-informed decisions that optimize business performance.**

Sensitivity and Specificity, Confusion Matrix at P Cutoff > 0.5

		True default status		
		No	Yes	
Predicted default status (cutoff $p > 0.5$)	No	TN = 484	FN = 11	$N^* = 495$
	Yes	FP = 2	TP = 3	$P^* = 5$
		$N = 486$	$P = 14$	

- **Sensitivity:** True positive rate = recall
 - $TP/P = 3 / 14 = 21.4\%$
- **Specificity:** True negative rate
 - $TN/N = 484 / 486 = 99.5\%$
- **False positive rate** (aka Type I error, 1 - Specificity)
 - $FP/N = 2/486 = 0.004\%$

Producing Confusion Matrix Using Formatted Results Data

- The `confusion_matrix()` function produces the confusion matrix
- We have to add index and column names to make it more legible

Confusion Matrix at cutoff = 0.30

	Pred 0 (No)	Pred 1 (Yes)
Actual 0 (No)	9735	0
Actual 1 (Yes)	264	1

Classification Report:

	precision	recall	f1-score	support
0	0.974	1.000	0.987	9735
1	1.000	0.004	0.008	265
accuracy			0.974	10000
macro avg	0.987	0.502	0.497	10000
weighted avg	0.974	0.974	0.961	10000

```
# probs from statsmodels:
y_prob = logit_fit.predict(X) # <-- not predict_proba

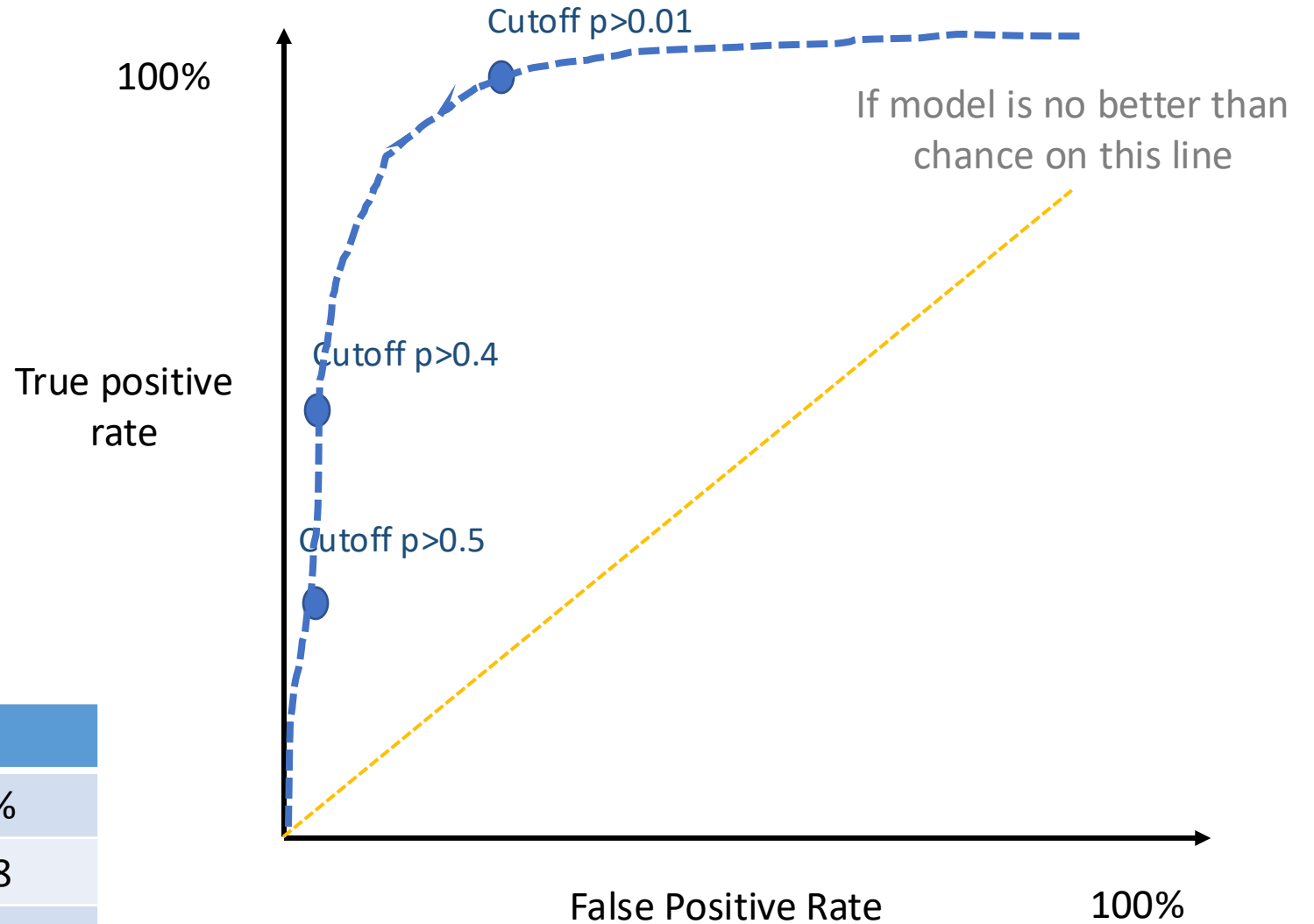
# classify at cutoff = 0.3
cutoff = 0.30
y_pred = (y_prob >= cutoff).astype(int)

# confusion matrix
cm = confusion_matrix(y, y_pred)
cm_df = pd.DataFrame(cm,
|         |         |         |         |         | index=["Actual 0 (No)", "Actual 1 (Yes)"],
|         |         |         |         |         | columns=["Pred 0 (No)", "Pred 1 (Yes)"])
print("Confusion Matrix at cutoff = 0.30")
print(cm_df)
```

Continuous Cutoff: ROC Curve

- Can we show consequences of FPs and FNs as we vary the cutoff probability to assign classes?
- Idea of a ROC (Receiver Operator Curve) plot

Cutoff	TPR	FPR
0.01	100%	22.6%
0.4	57%	0.008
0.5	21.4%	0.004%
0.6	21.4%	0.002%



ROC Curve

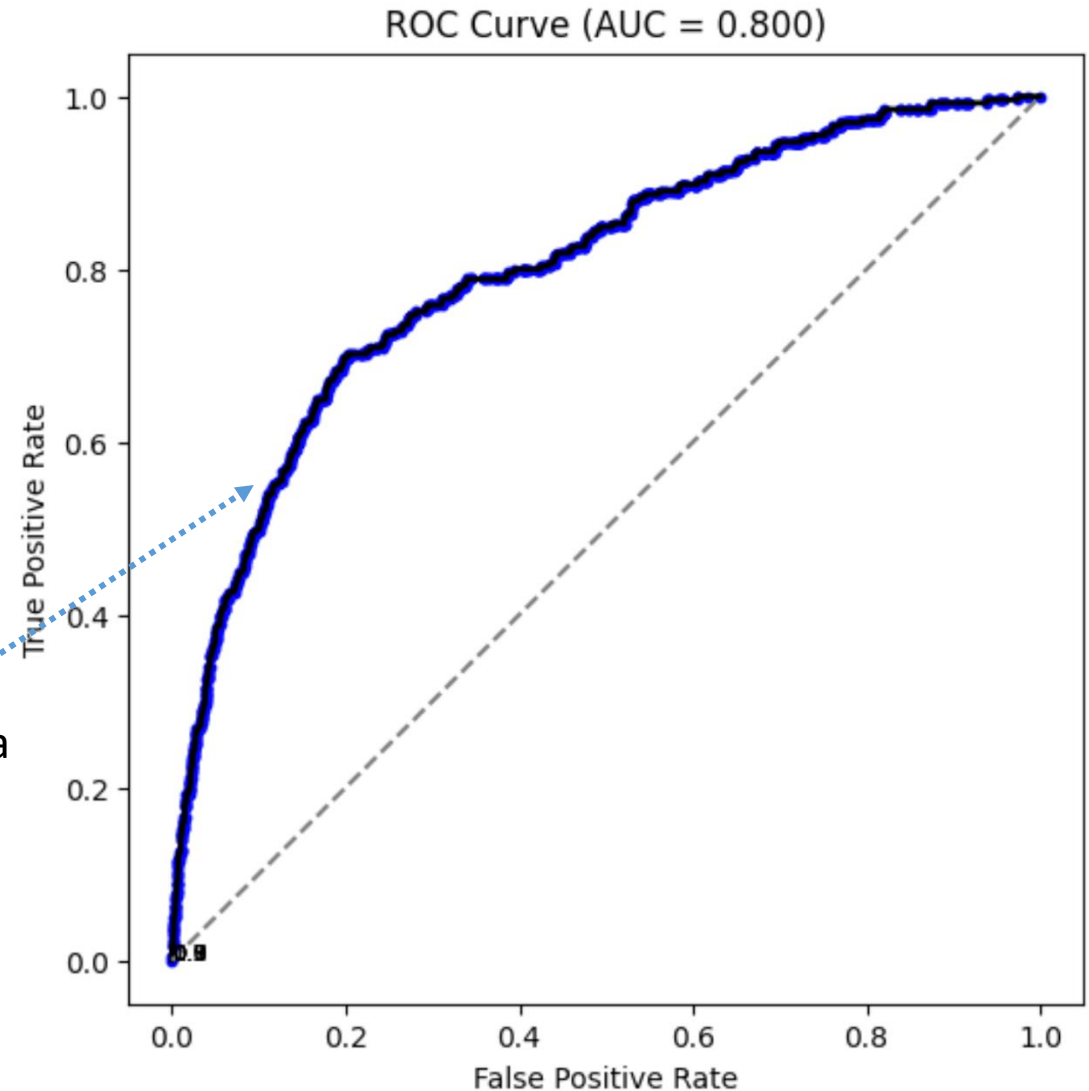
```
# true labels
y_true = df["default_binary"]

# predicted probabilities
y_scores = df["score"]

# compute ROC curve
fpr, tpr, thresholds = roc_curve(y_true, y_scores)

# compute AUC
auc = roc_auc_score(y_true, y_scores)
print("AUC:", auc)
```

- At a cutoff of 0.1, we get a true positive fraction of 0.5 and a false positive fraction of a very low number
- Better models lie up and to the left in the ROC plot
- AUC calculates how much total area is under a particular curve
- AUC of 0.8 is okay



Lab 2 — Logistic Regression in Python (Time permitting)

Data columns assumed: `default` (Yes/No), `student` (Yes/No), `balance`, `income`.

1) Estimate a logistic regression model

Fit a logistic regression of `default` on the predictors `income`, `student`, and `balance`.
Store this model as `logit_mod4`.

[]

2) Generate predicted probabilities

Use the fitted model to generate predicted probabilities (scores) for the training data.
Store them as a new column called `p_hat`.

Generate

+ Code

+ Markdown

Class 5 Summary

- Logit functions compress predictions to lie between 0 and 1, which are valid probabilities
- The logistic model models the outcome (Y) as the log odds ratio!
- Confusion matrices show the true/false positives/negatives.
- ROC plots measure the consequence on true positive fraction and false positive fraction for different cutoff probabilities
- Higher AUC scores mean a better ROC plot indicating a better model
- Classification, particularly logistic regression, is a decision-support tool that helps businesses predict and categorize outcomes based on data-driven probabilities.
- By modeling the likelihood of events such as customer churn, fraud detection, or loan default, classification enables managers to make proactive, risk-informed decisions that optimize business performance